

AgroVision AI

Offline Plant Disease Detection Using Deep Learning on Mobile

Final Exam Preparation Guide

Stack: Flutter | TFLite | EfficientNetB0 + MobileNetV2 + CBAM | PlantVillage | Python/Keras

1. Abstract

Plant diseases cause significant crop losses globally, yet early and accurate diagnosis remains inaccessible to smallholder farmers due to expensive expert consultation and unreliable internet connectivity in rural areas. This project presents AgroVision AI, an offline-first Android application that performs real-time plant disease classification entirely on-device using a custom dual-backbone deep learning model. The model combines EfficientNetB0 and MobileNetV2 feature extractors, enhanced with a Convolutional Block Attention Module (CBAM), trained on the PlantVillage dataset (87,848 images, 38 disease classes, 14 crop species). Test accuracy is approximately 98.3%. The 14.8 MB TFLite Float16 model runs offline on any Android 5.0+ device. The Flutter app supports 5 languages (EN, KN, HI, TA, TE), voice output via TTS, image quality validation, scan history, and an offline disease treatment library.

Key sentence: "AgroVision AI brings expert-level plant disease diagnosis offline, directly to a farmer's smartphone, with 98.3% accuracy and no internet dependency."

2. Methodology

2.1 Dataset

- PlantVillage dataset - 87,848 RGB leaf images (Kaggle, open-source)
- 38 classes: diseases + healthy, covering 14 crops (Tomato, Pepper, Grape, Apple, Corn, Potato...)
- Split: 80% Train (70,278) / 10% Validation (8,785) / 10% Test (8,785)
- Preprocessing: resize 224x224, normalize pixel = $(px / 127.5) - 1.0$ -> range [-1, 1]
- Augmentation (train only): random H/V flip, rotation +/-25 deg, zoom +/-20%, brightness +/-20%

2.2 Model Architecture - Dual Backbone + CBAM

Hybrid dual-backbone CNN with attention applied to each backbone output:

Component	Detail
Input	224 x 224 x 3 RGB image
EfficientNetB0	ImageNet pretrained, 5.3M params, MBConv blocks -> 1,280-d via GAP
CBAM #1	Channel + Spatial Attention on EfficientNet output
MobileNetV2	ImageNet pretrained, 3.4M params, Inverted Residuals -> 1,280-d via GAP
CBAM #2	Channel + Spatial Attention on MobileNet output
Concatenate	$1,280 + 1,280 = 2,560$ -d joint feature vector
Dense(512) + BN	BatchNorm + ReLU + Dropout(0.5)
Dense(256)	ReLU + Dropout(0.4)

Dense(38) Softmax	Output: probability over 38 disease classes
-------------------	---

2.3 CBAM - Convolutional Block Attention Module

- Channel Attention: GAP + GMP -> shared FC layers -> sigmoid -> weights which feature channels matter most
- Spatial Attention: avg + max across channels -> 7x7 Conv -> sigmoid mask -> highlights WHERE disease is on leaf
- Applied sequentially: Channel Attention first, then Spatial Attention
- Why: disease lesions are localized - attention suppresses background, forces focus on lesion region

2.4 Training Strategy - Two-Phase Transfer Learning

Phase	Epochs LR Frozen Layers Goal
Phase 1 - Head only	8 epochs lr=1e-3 Both backbones frozen Train head
Phase 2 - Fine-tune	4 epochs lr=5e-5 Top-20 backbone layers unfrozen Adapt

- Optimizer: Adam with Mixed Float16 precision (faster GPU training)
- Loss function: Categorical Cross-Entropy
- Callbacks: ModelCheckpoint (save best val_accuracy), EarlyStopping, ReduceLROnPlateau
- Platform: Kaggle NVIDIA T4 GPU | approx. 2 hours total training time

2.5 TFLite Conversion

- Best .keras checkpoint reloaded in pure Float32
- Converted with TFLite Float16 quantization (tf.lite.Optimize.DEFAULT)
- Output: final_model_float16.tflite - 14.8 MB | Input [1,224,224,3] | Output [1,38]

2.6 Flutter Application Architecture

- Presentation Layer: SplashScreen, Home, Scan, Result, History, Library, Settings (5-tab nav bar)
- Service Layer: TFLiteService, ImageQualityService, TtsService, HistoryService, DatabaseService, LocaleService
- Data Layer: Hive (scan history, offline), SQLite (treatment DB), TFLite model asset (on-device)
- 100% offline - zero network calls at any time

2.7 On-Device Inference Pipeline (6 Steps)

- Step 1: User captures image via camera or selects from gallery
- Step 2 - Quality Gate: file <=15 MB, dimensions >=100px, brightness avg 20-245, Laplacian variance >8 (blur), HSV green ratio >2% (leaf check)
- Step 3 - Preprocess: bake EXIF orientation, bicubic resize 224x224, normalize (px/127.5)-1.0 -> Float32List of 150,528 values
- Step 4 - Inference: Float32List -> TFLite interpreter.invoke() -> output [1,38] softmax probabilities - all on-device
- Step 5 - Post-process: argmax -> top class, confidence >=40% threshold, severity: <75%=Mild, 75-90%=Moderate, >=90%=Severe
- Step 6 - Result: disease name, confidence %, severity badge, symptoms/remedy/prevention, TTS readout, auto-save to history

3. Results Obtained

3.1 Model Performance

Metric	Value
Test Set Accuracy	approx. 98.3%
Model Size (Float16 TFLite)	14.8 MB
Number of Classes	38 (diseases + healthy)
Crop Species Covered	14 species
Total Dataset	87,848 images
Training Images	70,278
Training Platform	Kaggle NVIDIA T4 GPU
Inference	100% on-device - no internet required

3.2 Application Features Delivered

Feature	Implementation
Offline inference	TFLite model bundled in APK, zero network calls
Image quality gate	5 checks: size, blur, brightness, leaf detect, dimensions
Confidence threshold	Below 40% -> 'Not Recognized' (prevents false diagnoses)
Severity rating	Mild (<75%) / Moderate (75-90%) / Severe (>=90%)
Voice output TTS	EN, Kannada, Hindi, Tamil, Telugu
Scan history	All scans stored locally in Hive NoSQL (offline)
Disease library	38 diseases: symptoms, causes, remedy, prevention (SQLite)
Multilingual UI	Full app UI in 5 languages
Material 3 UI	Dark/light theme, animations, Hero transitions

3.3 Key Design Decisions

- Dual backbone: EfficientNetB0 (fine textures/spots) + MobileNetV2 (leaf structure) give complementary features, boosting accuracy vs. single backbone.
- CBAM: Lesions are localized. Attention forces focus on disease region, suppressing irrelevant background.
- Float16 quantization: Halves model size (29MB -> 14.8MB) with <0.5% accuracy loss, enabling small APK.
- Offline-first: Farmers in rural India have unreliable internet. App must work with zero connectivity.
- Image quality gate: Bad input produces meaningless predictions - gating prevents harmful misdiagnoses.

4. PPT Slide Outline (12 Slides, 10-15 Minutes)

Aim for 1 to 1.5 minutes per slide. Reference DIAGRAMS.md for Figures 1-5.

Slide 1: Title Slide

- AgroVision AI - Offline Plant Disease Detection Using Deep Learning on Mobile
- Your name, date, institution name
- Add app logo or a diseased leaf photo as visual

Slide 2: Problem Statement & Motivation

- Plant diseases cause 20-40% annual crop loss worldwide (FAO)
- Expert diagnosis is expensive, slow, and unavailable in rural areas
- Farmers in rural India have no reliable internet for cloud AI tools
- No existing affordable, offline, accurate diagnostic tool -> AgroVision AI solves this

Slide 3: Project Objectives

- Build a deep learning model with >95% accuracy on 38 plant disease classes
- Deploy as a lightweight TFLite model on Android (no GPU, no server required)
- Develop a fully offline Flutter app with multilingual support (5 languages)
- Provide actionable disease treatment information directly to farmers

Slide 4: System Architecture

- Show Fig. 1 from DIAGRAMS.md (Overall System Architecture)
- 4 layers: Presentation (Flutter screens), Service (logic), Data (Hive/SQLite/model), ML Engine
- Key message: Everything runs on the phone. No server, no internet required.

Slide 5: Dataset

- PlantVillage - 87,848 images | 38 classes | 14 crop species
- Split: 80% Train (70,278) / 10% Val / 10% Test (8,785 each)
- Preprocessing: resize 224x224, normalize (px/127.5)-1.0 -> range [-1, 1]
- Augmentation (train only): flip, rotation, zoom, brightness variations
- Tip: add a 3x3 grid of sample leaf images showing different disease classes

Slide 6: Model Architecture - Dual Backbone + CBAM

- Show Fig. 2 from DIAGRAMS.md (CNN Model Architecture)
- EfficientNetB0 (5.3M params) + MobileNetV2 (3.4M params) - both ImageNet pretrained
- CBAM attention applied to each backbone output independently
- Concatenate: 1,280 + 1,280 = 2,560-d joint feature vector
- Head: BatchNorm -> Dense(512, ReLU) -> Dense(256, ReLU) -> Dense(38, Softmax)

Slide 7: CBAM - Attention Mechanism

- Channel Attention: GAP + GMP -> FC layers -> sigmoid -> weights which channels matter
- Spatial Attention: avg+max across channels -> 7x7 Conv -> sigmoid mask -> WHERE is disease
- Applied sequentially: Channel first, then Spatial
- Why: disease lesions are localized - attention removes background, focuses on lesion

Slide 8: Training Pipeline

- Show Fig. 3 from DIAGRAMS.md (Training Pipeline)
- Phase 1 (8 epochs, lr=1e-3): backbones frozen, train classification head only
- Phase 2 (4 epochs, lr=5e-5): top-20 layers per backbone unfrozen for fine-tuning
- Callbacks: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
- Kaggle T4 GPU | Mixed Float16 | ~2 hours | converted to TFLite Float16 -> 14.8 MB

Slide 9: On-Device Inference Pipeline

- Show Fig. 4 from DIAGRAMS.md (Mobile App Inference Pipeline)
- Steps 1-2: Image input -> 5-check quality gate (reject blurry/dark/non-leaf images)
- Step 3: Preprocess - resize 224x224, normalize -> Float32List of 150,528 values
- Step 4: TFLite interpreter.invoke() runs forward pass entirely on-device
- Steps 5-6: Argmax + threshold + severity -> Result screen + TTS + auto-save history

Slide 10: App Screenshots / Demo

- Scan screen: camera + gallery buttons, pulse animation, quality tips
- Result screen: disease name, confidence bar, severity badge (Mild/Moderate/Severe)
- Disease library: searchable grid of 38 disease cards with treatment bottom sheet
- History screen: past scan cards with filter chips (All / Healthy / Disease)
- Settings: language selector (EN, KN, HI, TA, TE) and TTS toggle

Slide 11: Results & Performance

- Test Accuracy: approx. 98.3% on 8,785-image held-out test set
- Model Size: 14.8 MB Float16 TFLite - fits inside the app bundle
- 38 disease classes across 14 crop species
- 5 language support with full TTS voice output
- 0% internet dependency - works in airplane mode

Slide 12: Conclusion & Future Work

- Built offline plant disease detector achieving ~98.3% accuracy
- Runs on any Android 5.0+ device with no internet and no cost
- Multilingual TTS makes it accessible to non-literate farmers
- Future: expand to Indian crop diseases, INT8 quantization, lesion segmentation, iOS port

5. Likely Exam Questions & Answers

Q1: Why did you use two backbones instead of one?

Two backbones extract complementary features. EfficientNetB0 focuses on fine-grained textures (disease spots, color change) while MobileNetV2 captures structural patterns (leaf morphology). Concatenating both gives a richer 2,560-d representation than either alone, improving accuracy.

Q2: What is CBAM and why use it?

CBAM (Convolutional Block Attention Module) applies channel-wise then spatial attention. Channel attention learns which feature maps matter most; spatial attention highlights where on the leaf the disease is. Since disease occupies a small localized region, attention suppresses background and improves detection focus.

Q3: Why TFLite and not a cloud server?

Target users are farmers in rural India with no reliable internet. Cloud inference would make the app unusable in the field. TFLite enables fully offline, low-latency inference on any Android 5.0+ device with zero network dependency.

Q4: What is Float16 quantization? Does it hurt accuracy?

Float16 stores weights in 16-bit floating point instead of 32-bit, halving model size from ~29 MB to 14.8 MB. Accuracy loss is negligible (typically <0.5%). It also speeds up inference on hardware with native Float16 support.

Q5: Why do you have a confidence threshold?

A misdiagnosis is harmful - a farmer might apply the wrong pesticide. If the model confidence is below 40%, we show 'Not Recognized' instead of a potentially wrong answer, prompting the user to retake the photo or consult an expert.

Q6: What is transfer learning and why use it?

Transfer learning reuses a model pretrained on ImageNet (1.2M images, 1000 classes). The backbone already detects edges, textures, and shapes. We fine-tune it for plant disease classification with far less data and compute than training from scratch.

Q7: What is two-phase training?

Phase 1 freezes the backbone and trains only the classification head - fast convergence, avoids destroying pretrained weights. Phase 2 unfreezes the top backbone layers and fine-tunes them at a very low lr (5e-5) to adapt to plant leaves. More stable than unfreezing all at once.

Q8: What does the image quality service do?

5 checks run before inference: file <=15 MB, dimensions >=100px, brightness avg 20-245 (not dark/overexposed), Laplacian variance >8 (not blurry), HSV green ratio >2% (actually a leaf). Any failure prompts the user to retake the photo.

Q9: How does the app show disease info offline?

All 38 disease treatments (symptoms, causes, remedy, prevention) are stored in an SQLite database bundled inside the APK at install time. No internet call is ever made.

Q10: What is the PlantVillage dataset?

PlantVillage is an open-source dataset of 87,848 labeled leaf images covering 38 disease categories across 14 crop species. It is the standard benchmark for plant disease classification research.

6. Tech Stack Quick Reference

Component	Technology / Detail
Mobile Framework	Flutter 3.41 / Dart 3.11 Android arm64
ML Runtime	tflite_flutter ^0.12.1 On-device, fully offline
Model	EfficientNetB0 + MobileNetV2 + CBAM 38-class Softmax
Model File	Float16 TFLite 14.8 MB Input [1,224,224,3] Output [1,38]
Dataset	PlantVillage 87,848 images 38 classes 14 crops
Training	Keras 3 / TensorFlow Kaggle T4 GPU Mixed Float16
Scan History DB	Hive (NoSQL, offline)
Disease Treatment DB	SQLite 38 diseases, symptoms + remedy + prevention
Voice Output TTS	flutter_tts ^4.1.0 EN, Kannada, Hindi, Tamil, Telugu
UI Design	Material 3 Dark/light theme Hero animations
Min Android	API 21 (Android 5.0+)

AgroVision AI | Exam Prep | Good luck!